CODE SPORT

In this month's column, we discuss some of the basic questions in machine learning and text mining.

As we have been doing over the last couple of months, we will continue to discuss computer science interview questions, focusing on topics in machine learning and text analytics. While it is not necessary that one should know the mathematical details of the state-of-art algorithms for different NLP techniques, it is assumed that readers are familiar with the basic concepts and ideas in text analytics and NLP. For example, no one ever needs to implement back propagation code for a deep layered neural network, since this is provided as a utility function by the neural network libraries for different cost functions. Yet, one should be able to explain the concepts and derive the basic back propagation equations on a simple neural network for different loss functions, such as cross-entropy loss function or root mean square loss function, etc.

It is also important to note that many of the questions are typically oriented towards practical implementation or deployment issues, rather than just concepts or theory. So it is important for the interview candidates to make sure that they get adequate implementation experience with machine learning/NLP projects before their interviews. For instance, while most textbooks teach the basics of neural networks using a 'sigmoid' or 'hyperbolic tangent' (tanh) function as the activation function, hardly anyone uses the 'sigmoid' or 'tanh' functions in real-life implementations. In practice, the most commonly used activation function is the RELU (rectified linear) function in inner layers and, typically, softmax classifier is used in the final output layer.

Very often, interviewers weed out folks who are not hands-on, by asking them about the activation functions they would choose and the reason for their choices. (Sigmoid and hyperbolic tangent functions take a long time to learn and hence are not preferred in practice since they slow down the training considerably.)

Another popular question among interviewers is about mini-batch sizes in neural network training. Typically, training sets are broken into mini-batches and then cost function gradients are computed on each mini-batch, before the neural network weight parameters are updated using the computed gradients. The question often posed is: Why do we need to break down the training set into mini-batches instead of computing the gradient over the entire training set? Computing the gradients over the entire training set before doing the update will be extremely slow, as you need to go over thousands of samples before doing even a single update to the network parameters, and hence the learning process is very slow. On the other hand, stochastic gradient descent employs a mini-batch size of one (the gradients are updated after processing each single training sample); so the learning process is extremely rapid in this case.

Now comes the tricky part. If stochastic gradient descent is so fast, why do we employ mini-batch sizes that are greater than one? Typical mini-batch sizes can be 32, 64 or 128. This question will stump most interviewees unless they have hands-on implementation experience. The reason is that most neural networks run on GPUs or CPUs with multiple cores. These machines can do multiple operations in parallel. Hence, computing gradients for one training sample at a time leads to non-optimal use of the available computing resources. Therefore, mini-batch sizes are typically chosen based on the available parallelism of the computing GPU/CPU servers.

Another practical implementation question that gets asked is related to applying dropout techniques. While most of you would be familiar with the theoretical concept of drop-out, here is a trick question which interviewers frequently ask. Let us assume that you have employed a uniform dropout rate of 0.7 for each inner layer during training on a 4-layer feed forward neural network. After training the network,